

SCSIT

IV Semester

Database

Exc. 4

- **Database – Table – Column – Touple - Field**
 - **Relational Algebra - σ - Π - ρ**
 - **Introduction to SQL**
-

Some terms

ER Diagram  Relational DB model

Entity/ n-m Relation  Table

Attribute  Column

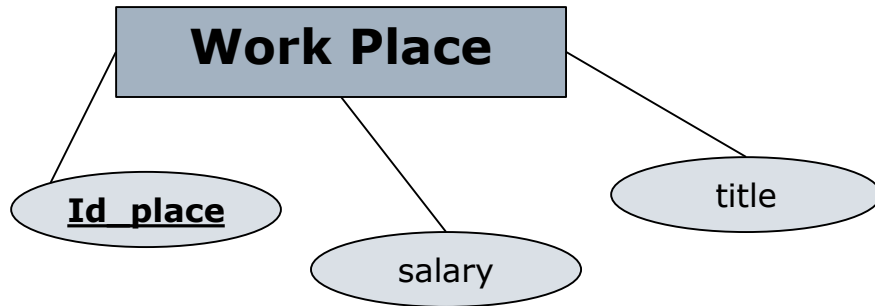
Instance  Touple (row)

Attribute value  Table field value

Example

Entity: WORKPLACE

- Id_place
- Title
- Salary



Instances

Work Place (1, "Manager", 45000)
 Work Place (2, "DBA", 27000)
 Work Place (3, "Analyst", 30000)
 Work Place (4, "Programmer", 25000)
 Work Place (5, "Secretary", 10000)

Table: WorkPlace

Id: PK, Autonumber, Not null

Title:Text(50), Not null

Salary:Number

WorkPlace

* Id
 Title
 Salary

Id	Title	Salary
1	Manager	45000
2	DBA	27000
3	Analyst	30000
4	Programmer	25000
5	Secretary	10000

Table, Column, Tuple, Field

Table →

TUPLE →

column: has values for ONE attribute

field

Field= One value for an attribute in specific row (tuple)

Id	title	Salary
1	Manager	45000
2	DBA	27000
3	Analyst	30000
4	Programmer	25000
5	Secretary	10000

Exercise: Table, column, tuple, field

What of the following is table, tuple, column or field value

Item	T/C/To/F
20000	Field – value of an attribute
Member_id	Column
Michael	Field
3, “ET”, 25-01-2009	Touple -row
4, Skopje	Touple - row
Macedonia	Field
Organization	Column (it may also be a table)
Client	Table

What can we do with DB Data?

- ☐ How to find and use DB data?
- ☐ Ex.
 - ☐ Find all salaries from WorkPlace?
 - ☐ Get all titles from WorkPlace, but the result column should be named as “position”?
 - ☐ Get all data for the row with ID=3
 - ☐ Get all WorkPlace rows that have salary value below 30000

Query languages

- QL are languages which are used for user requests information from the database. They are typically at higher level than programming languages.
- QL can be:
 - **Procedural:** the user gives instructions to the system how to perform an operation on the database. Database will process the request and will return the requested information.
 - **Nonprocedural:** user specifies the requested information without giving the procedure how to obtain it.
- Complete QL includes possibilities for Updating, inserting and deleting tuples from DB Tables, as well as commands for tuple and table structure modifications and modification

Relational algebra

- ❑ Procedural QL
- ❑ **Query** is Rel. Algebra statement created by the DB User as a DB request, to whom DB will return response (i.e. result data)
- ❑ Example:

$(x+y)*y \quad x,y \in \mathbb{N}$ – Math. algebra

$\pi_{(PNO, PNAME, BUDGET)} (\sigma_{(BUDGET > 30000)}(PROJ) \times \pi_{(PNO, ENO)}(EMP) =$

DB Algebra

Operations

☐ Basic

☐ projection (π)

☐ rename (ρ)

☐ selection (σ)

☐ Cartesian product

☐ Union

☐ Substraction

☐ Functions (math, logical, aggregate):

☐ +, -, /, *, and, or, Xor, not, count, min, max, len ...

☐ Derived

☐ Intersection

☐ Natural *join*

☐ Division

☐ Assign

Selection - σ

□ Result: Rows (touples) that fulfilled the given condition

□ **Ex.**

1. Select all tuples from WorkPlace

$\sigma(\text{workplace})$

Id	Title	Salary
1	Manager	45000
2	DBA	27000
3	Analyst	30000
4	Programmer	25000
5	Secretary	10000

2. Select all tuples from WorkPlace salary below 30000

$\sigma_{\text{salary} < 30000}(\text{Workplace})$

ID	Title	Salary
1	Manager	45000
2	DBA	27000
3	Analyst	30000
4	Programmer	25000
5	Secretary	10000



Id	Title	Salary
2	DBA	27000
4	Programmer	25000
5	Secretary	10000

Selection Exercise

1. Get all data from tuple with id=3
2. Get all data for those workplaces that have salary less than 10000 or more than 35000
3. Get all data from workplace with title Secretary and even ID.
4. Find maximal, minimal and average salary.

Answers

1. $\sigma_{id=3}$ (workplace)
2. $\sigma_{(salary \leq 10000 \text{ or } salary \geq 35000)}$ (workplace)
3. ?
4. ?

What are the returned result sets?

Projection– Π

□ Returns field values from column

□ **Ex.**

1. Select all salaries

$\Pi_{\text{salary}}(\text{workplace})$



Salary
45000
27000
30000
25000
10000

2. Select salaries and titles from workPlace.

$\Pi_{\text{salary}, \text{title}}(\text{workplace})$

Id	Title	Salary
1	Manager	45000
2	DBA	27000
3	Analyst	30000
4	Programmer	25000
5	Secretary	10000



Title	Salary
Manager	45000
DBA	27000
Analyst	30000
Programmer	25000
Secretary	10000

Projection exercise

1. Get all titles and ID's
2. Get all title and salaries
3. Using projection, Get all data from workplace

Answers

1. $\Pi_{\text{title, id}}(\text{workplace})$

2. ?

3. ?

What are the returned result sets?

Rename (*aliasing*)– $\rho_{a / b}(R)$

- Rename table (R) column a into b, but only in the result set
- **Ex.** Select all data for titles, but rename the result column into “Position”

$\rho_{title / \text{“Position”}}(\text{workplace})$

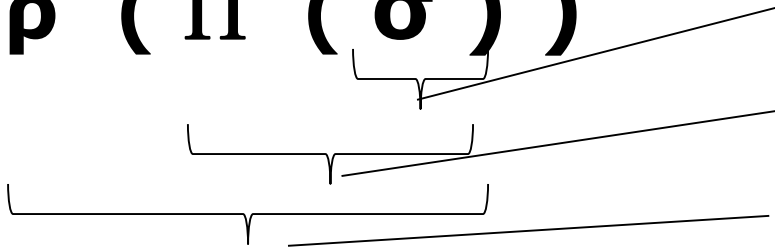
Id	Title	Salary
1	Manager	45000
2	DBA	27000
3	Analyst	30000
4	Programmer	25000
5	Secretary	10000



Position
Manager
DBA
Analyst
Programmer
Secretary

σ , Π and ρ priority

□ $\rho (\Pi (\sigma))$

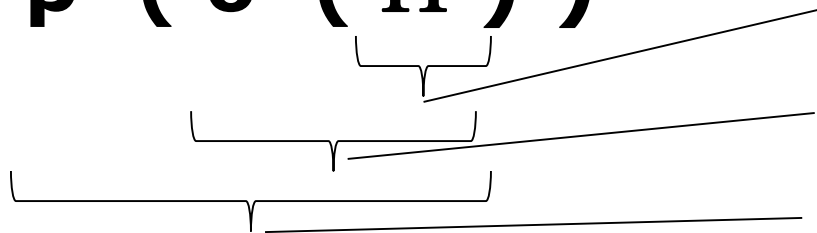


1. Tuple selection

2. Column projection

3. Column rename

□ $\rho (\sigma (\Pi))$



1. Column projection

2. Tuple selection

3. Column rename

Aliasing (renaming) is done after all other operations are done.

Combining σ , Π and ρ c

Id_place	Title	Salary	Sector	Phone	Vehicle	LAPTOP
1	Manager	45000	Marketing	1	1	1
2	DBA	27000	IT	0	0	0
3	Analyst	30000	IT	1	0	1
4	Programmer	25000	IT	0	0	0
5	Secretary	10000	Administration	0	0	0
6	Doorman	7000	Administration	0	0	0
7	Accountant	12000	Finance	1	0	0
8	Treasurer	13000	Finance	1	0	0
9	Driver	9000	Service	1	1	0
10	Event manager	23000	Marketing	1	1	1

Get title and sector for salaries smaller than 12000. Rename sector into *department*

Combining σ , Π and ρ c

- Get title and sector for salaries smaller than 12000.
Rename sector into *department*

Id_place	Title	Salary	department	Phone	Vehicle	LAPTOP
1	Manager	45000	Marketing	1	1	1
2	DBA	27000	IT	0	0	0
3	Analyst	30000	IT	1	0	1
4	Programmer	25000	IT	0	0	0
5	Secretary	10000	Administration	0	0	0
6	Doorman	7000	Administration	0	0	0
7	Accountant	12000	Finance	1	0	0
8	Treasurer	13000	Finance	1	0	0
9	Driver	9000	Service	1	1	0
10	Event manager	23000	Marketing	1	1	1

Result

Title	Department
Secretary	Administration
Doorman	Administration
Driver	Service

$\rho_{\text{sector/ department}} (\sigma_{\text{salary} < 12000} (\Pi_{\text{title, sector}} (\text{work_place})))$

=

$\rho_{\text{sector/ department}} (\Pi_{\text{title, salary}} (\sigma_{\text{salary} < 12000} (\text{work_place})))$

Exercises

- ❑ Find all sectors that have phone.
- ❑ Find all titles for work places that have salary above 25000 and also have laptop or phone.
- ❑ Get all touples (data rows) that don't have neither laptop nor phone nor vehicle.
- ❑ Find all salaries in IT Sector. Rename the record set column into "IT salaries"
- ❑ Find the title for the workplace that has maximal salary, but doesn't have neither phone nor laptop
- ❑ Calculate 1/3-th of the salaries from Marketing

SQL = Standard Query Language

- ❑ **Standard language for creation of nonprocedural queries. These queries serve for**
 - Creation of database object/components
 - Data manipulation and processing
 - Defining grant and deny access privileges for DB users

SQL can be divided to

- ❑ Data Definition Language (**DDL**) for DB Schema declaration
 - ❑ Creation of tables
 - ❑ Defining constraints (PK, relations, Indexes etc.)
- ❑ Data Manipulation Language (**DML**) for DB data manipulation, insertion, update and delete
- ❑ Data Control Language (**DCL**) for audit control

DML SQL:

Basic operations

- ❑ Data retrieving (no changes in DB data or definition)

 - ❑ **SELECT**

- ❑ Permanent data insert

 - ❑ **INSERT INTO**

- ❑ Permanent data change

 - ❑ **UPDATE**

- ❑ Permanent data removing from DB

 - ❑ **DELETE**

Transact-SQL functions

- ❑ Arithmetical (+, -, *, /)
- ❑ Column concatenation ||
- ❑ null value (NVL(column, default value))
- ❑ operators for comparison (>, <, =<, >=, =)
- ❑ Logical operator (not, !, and, or, ())
- ❑ SQL operators for comparison (BETWEEN, HAVING)
- ❑ Joker sings (% , _ , *)
- ❑ String and Char functions
- ❑ Number functions
- ❑ DateTime functions
- ❑ Conversion functions
- ❑ Aggregate functions

Their usage is mainly in SELECT statements

SELECT

Basic syntax for **ONE TABLE** data manipulation

SELECT [**DISTINCT**] {*, *column* [[**AS**] *alias*], ...}

FROM *table*

[**WHERE** *condition(s)*]

[**GROUP by** grouping statement]

[**HAVING** group condition(s)]

[**ORDER by** column(s) [ASC/DESC]]

SELECT examples (1/2)

SELECT GETDATE()

SELECT * FROM WorkPlace

SELECT salary, salary+5*salary/100 AS Sal_Increasing
FROM WorkPlace
WHERE SALARY IS NOT NULL
ORDER BY SALARY DESC

SELECT TITLE, LAPTOP, VEHICLE,PHONE
FROM WORK_PLACE
WHERE SALARY BETWEEN 10000 AND 20000

SELECT EXAMPLES (2/2)

```
SELECT LOWER(TITLE) AS PROFESSION,*  
FROM WORK_PLACE  
WHERE UPPER(SECTOR) like '%IN%'
```

```
SELECT MIN(SALARY), MAX(SALARY)  
FROM WORK_PLACE
```

```
SELECT *  
FROM WORK_PLACE  
WHERE SALARY < (SELECT MAX(SALARY)/3  
                FROM WORK_PLACE)
```

SQL VS. Relational Algebra

Operation	Rel. algebra	SQL
Selection	$\sigma_{\text{condition}}(\text{table})$	SELECT * FROM <i>table</i> WHERE condition
Projection	$\pi_{A,B,\dots,S}(\text{table})$	SELECT DISTINCT A,B,...S FROM <i>table</i>
Rename	$\rho_{A/A1}(\text{table})$	SELECT A as A1 FROM <i>table</i>
Combination	$\rho_{A/A1}(\pi_{A,B,C}(\sigma_{\text{cond.}}(\text{table})))$	SELECT A as A1, B, C FROM <i>table</i> WHERE cond

Translate Rel. Algebra into SQL queries

1. Find all touples with salary under 20000

$\sigma_{salary < 20000}(\text{work_place})$

SELECT *

FROM WORK_PLACE

WHERE SALARY < 20000

Translate Rel. Algebra into SQL queries

2. Find all tuples for work places from IT or Marketing sector that have laptop.

$\sigma_{(sector='IT' \text{ or } sector='MARKETING') \text{ and } laptop=1}$ (WORK_PLACE)

SELECT *

FROM WORK_PLACE

WHERE LAPTOP=1 AND

(SECTOR = 'IT' OR SECTOR='MARKETING')

Logic truth tables

Op.1	Op.2	And
T	T	T
T	F	F
F	T	F
F	F	F

Op.1	Op.2	OR
T	T	T
T	F	T
F	T	T
F	F	F

Op.1	NOT
T	F
F	T

Op.1	Op.2	XOR
T	T	F
T	F	T
F	T	T
F	F	F

Write SQL queries for

1. Get all salaries from **workplace** ?
2. Get all titles from **workplace**. Rename the result set column into *Position*
3. Get all data from **workplace** tuple (row) with id=3
4. Get all data from **workplace** for tuples with salary smaller than 30000
5. Get all data from **work_place** for people that have salary smaller than 35000 and bigger than 10000 .
6. Get all title and ID numbers form **work_place**
7. Get all data from **work_place** for tuples that have even ID and title Secretary
8. Find all **work_place** sectors that have phone

Write SQL queries for

1. Find work places that have salary below 25000 and also have laptop or phone
2. Find maximal salary from **work_place** and rename the result set into MAXIMUM_SALARY
3. Find tuple from **work_place** that don't have laptop, phone or vehicle.
4. Find salaries in IT sector. Rename the result set column into "IT_Salaries"
5. Find Maximal salary in finance sector. Rename the result into MAXIMUM_FIN
6. Find work place title for work place that has maximal salary but doesn't have phone nor laptop.
7. Find 1/3th from the Marketing salaries. (1/3th of their sum)